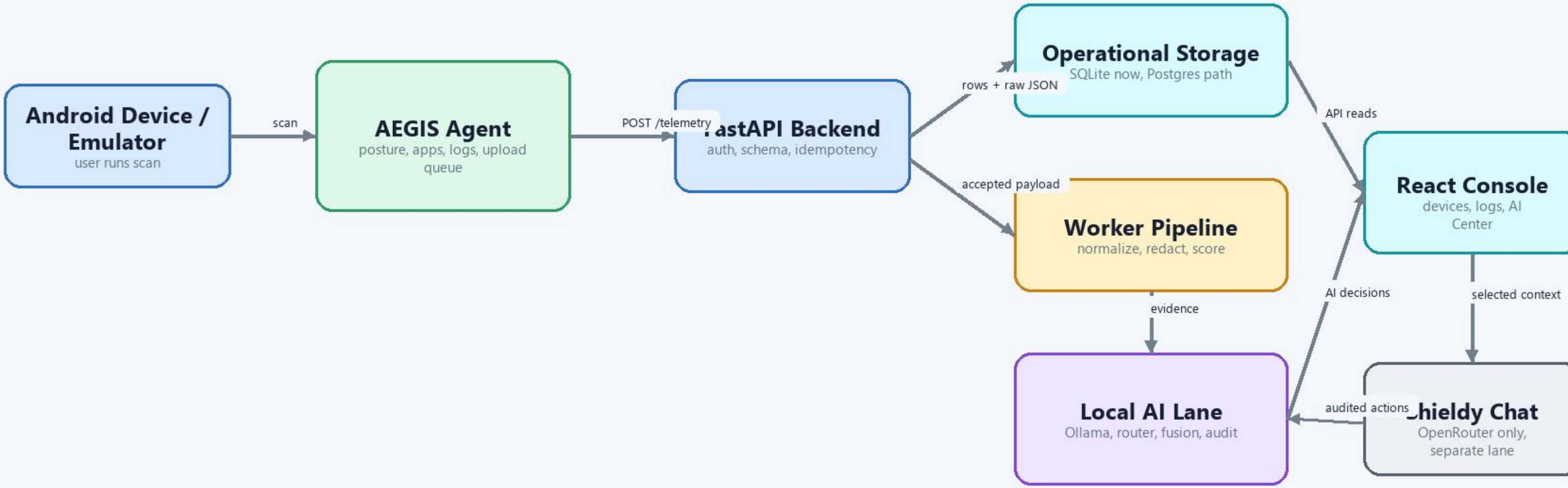
Full System Architecture

Android telemetry, backend processing, local AI analysis, and analyst console



Current runtime

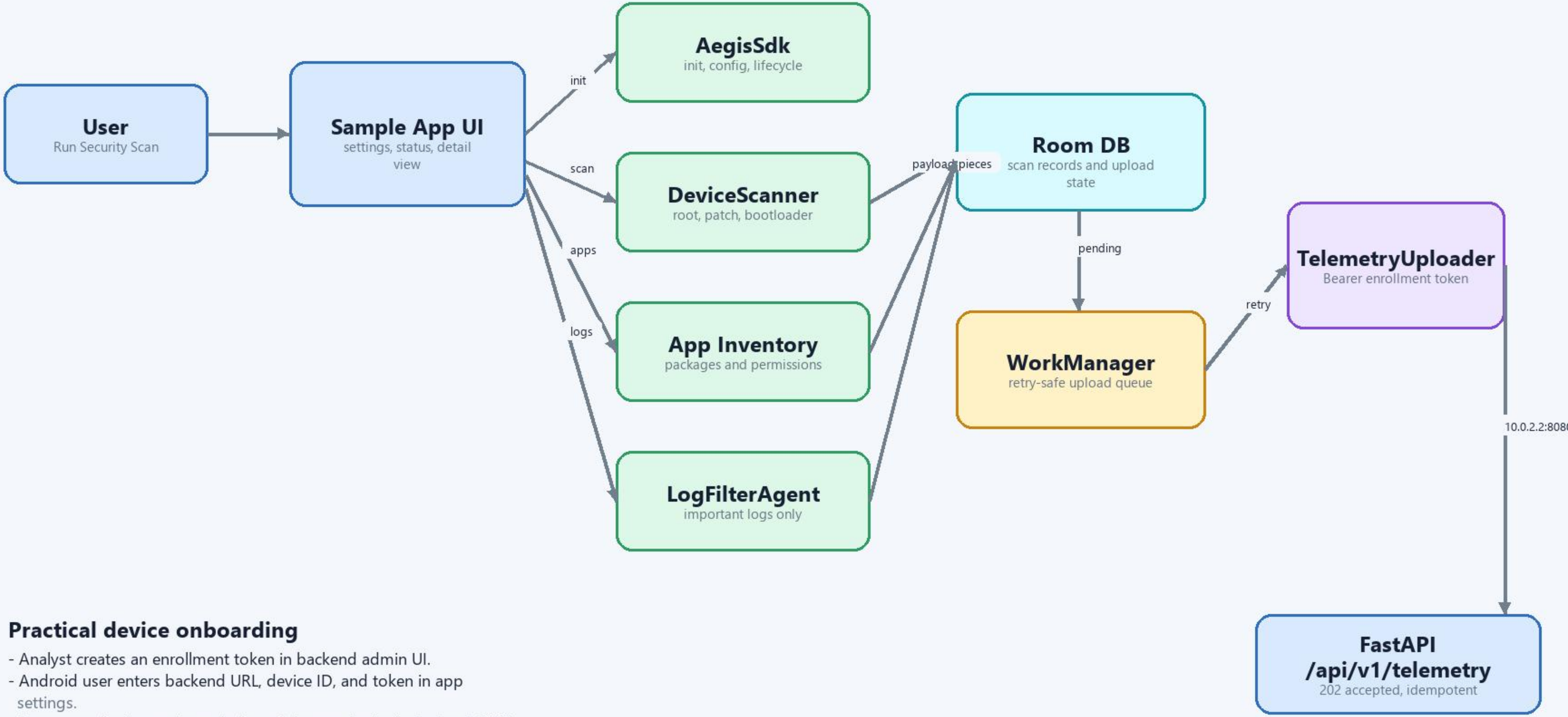
- FastAPI on 127.0.0.1:8080.
- React/Vite console on 127.0.0.1:5173.
- Ollama local models on 127.0.0.1:11434.
- OpenRouter is limited to Shieldy Chat and is not used for automated scoring.

Safety boundary

- Backend validation and deterministic rules run before AI.
- AI outputs must be JSON with evidence references.
- Invalid or slow model output falls back to deterministic/stub behavior.

Android Agent Architecture

How the sample app collects evidence and uploads telemetry without user code changes

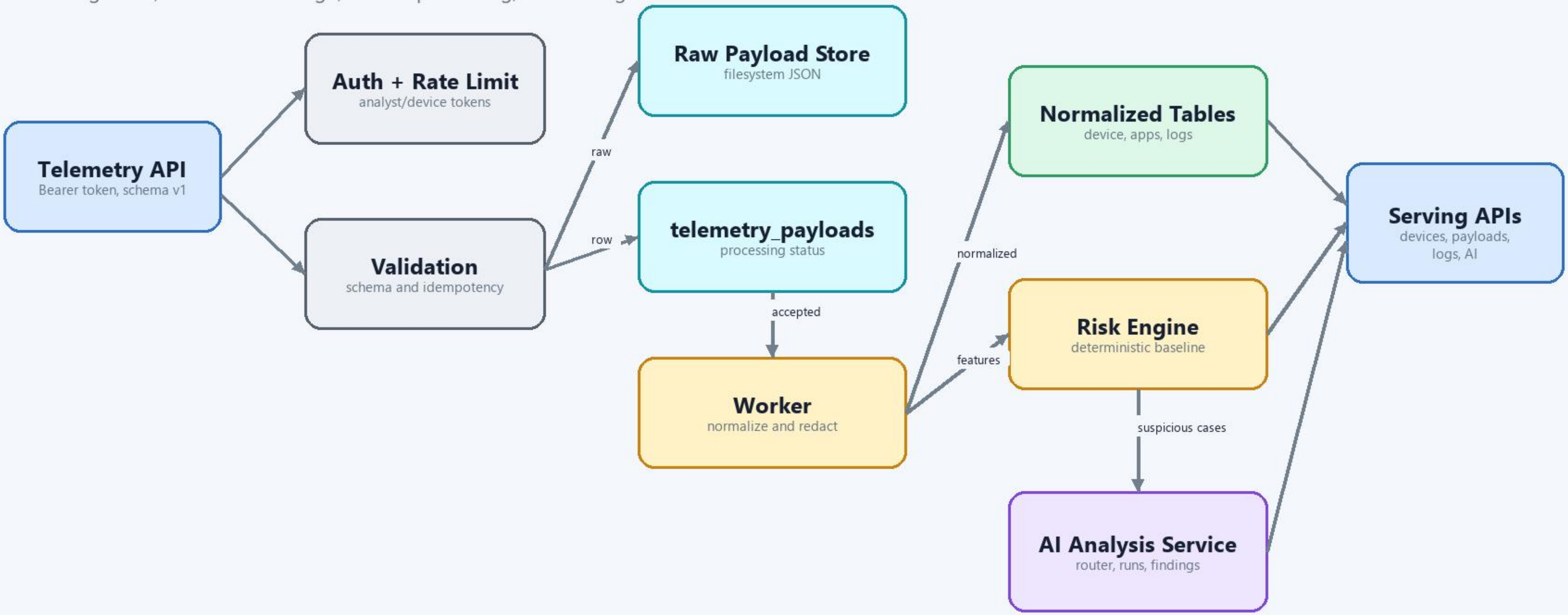


Practical device onboarding

- Analyst creates an enrollment token in backend admin UI.
- Android user enters backend URL, device ID, and token in app settings.
- No app code change is needed to add a new device in the local MVP.

Backend Architecture

FastAPI ingestion, normalized storage, worker processing, and serving APIs



Key backend tables

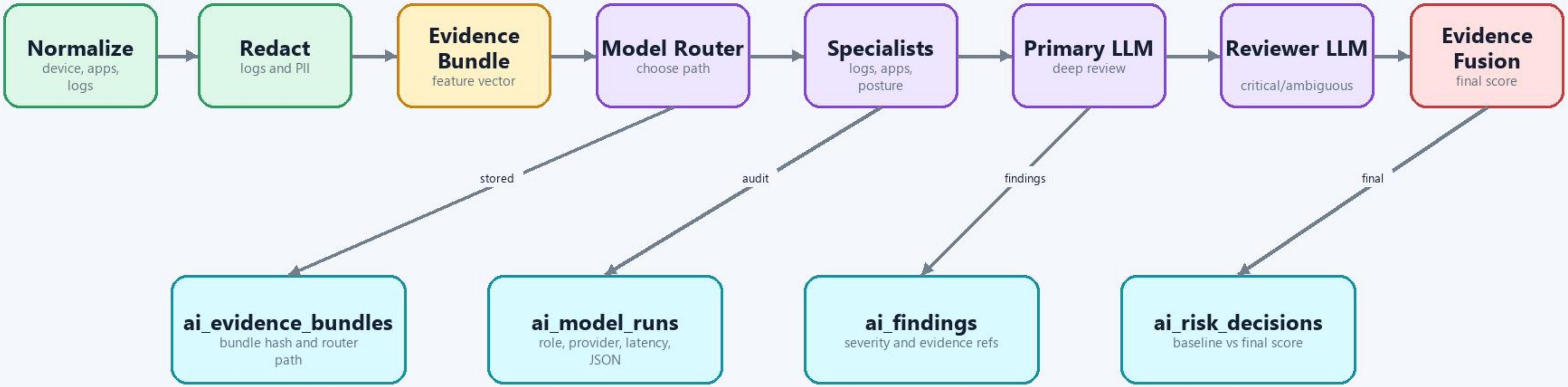
- telemetry_payloads, device_reports, app_inventory_current, important_logs.
- risk_assessments stores deterministic baseline and displayed score.
- ai_evidence_bundles, ai_model_runs, ai_findings, ai_risk_decisions make the AI lane auditable.

Reliability choices

- Raw JSON is stored before processing.
- Duplicate payload_id returns 202 without duplicate rows.
- AI timeout or invalid JSON does not block telemetry processing.

AI Analysis Architecture

Real automated analysis lane, separate from the chatbot



Router paths

- rules_specialists: low or simple evidence uses cheap specialist checks plus fusion.
- suspicious_primary: suspicious apps, logs, or medium/high baseline add primary LLM.
- critical_primary_reviewer: rooted, critical, or ambiguous evidence adds reviewer LLM.

Model contracts

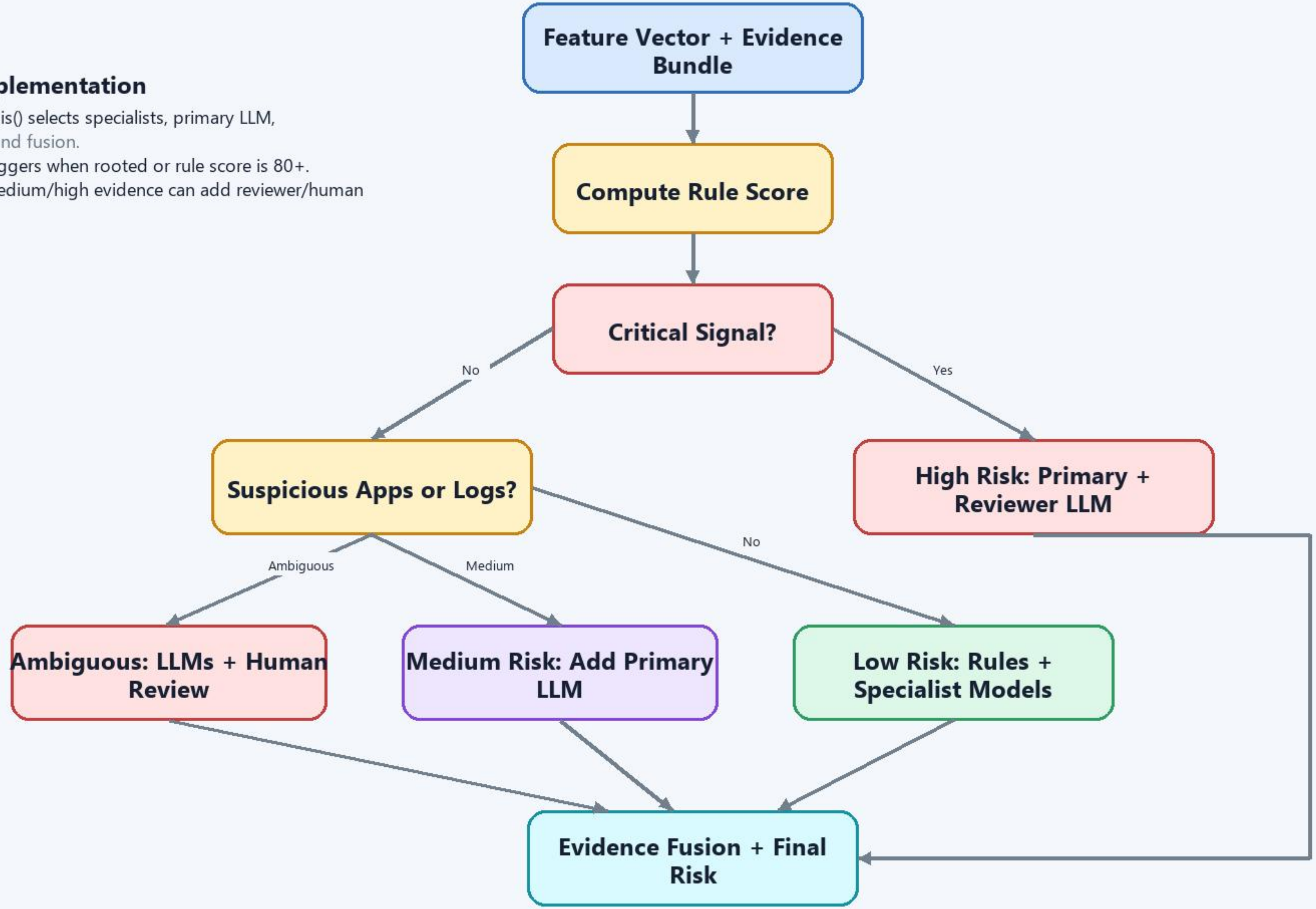
- Every model returns structured JSON only.
- Every finding must include evidence_refs from the bundle.
- Ollama can be replaced behind LocalAnalyzerProvider without changing backend APIs.

AI Model Router Decision Chart

Choose the cheapest reliable model path for each telemetry payload

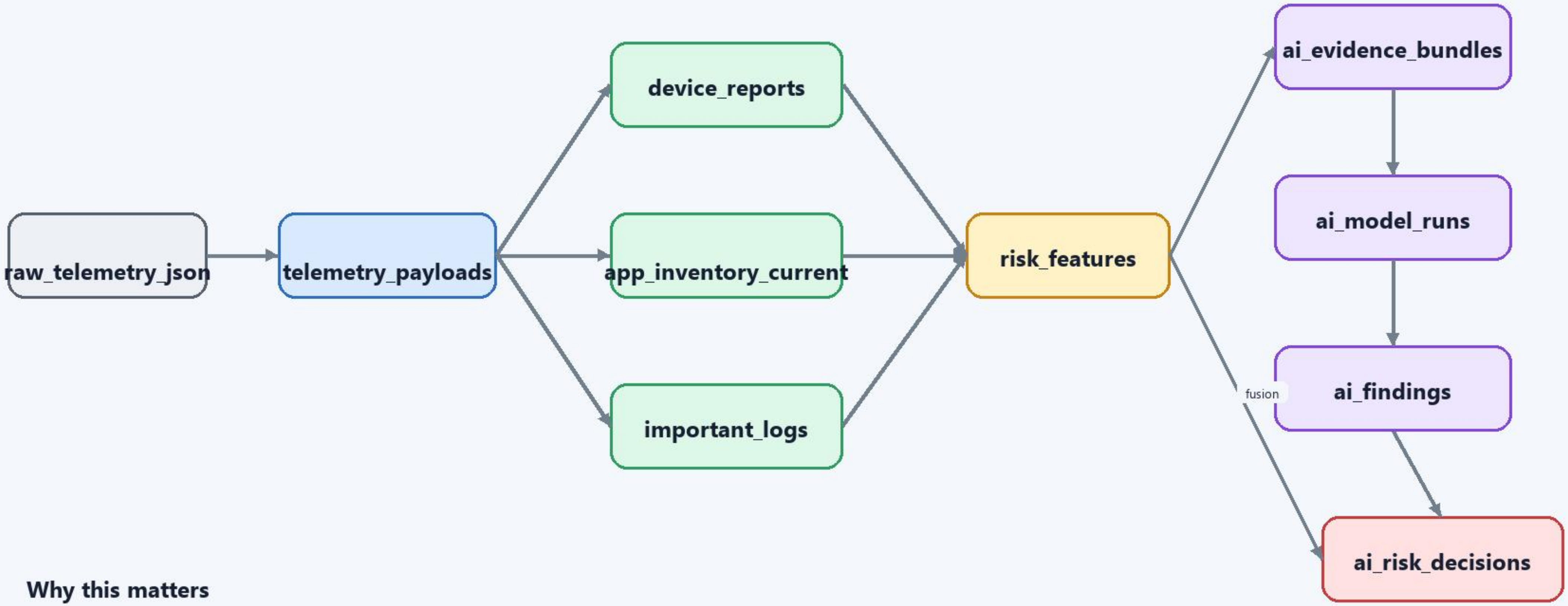
Current implementation

- route_ai_analysis() selects specialists, primary LLM, reviewer LLM, and fusion.
- Critical path triggers when rooted or rule score is 80+.
- Ambiguous medium/high evidence can add reviewer/human review.



AI Evidence And Storage Lineage

Trace every finding back to raw telemetry, features, model runs, and feedback

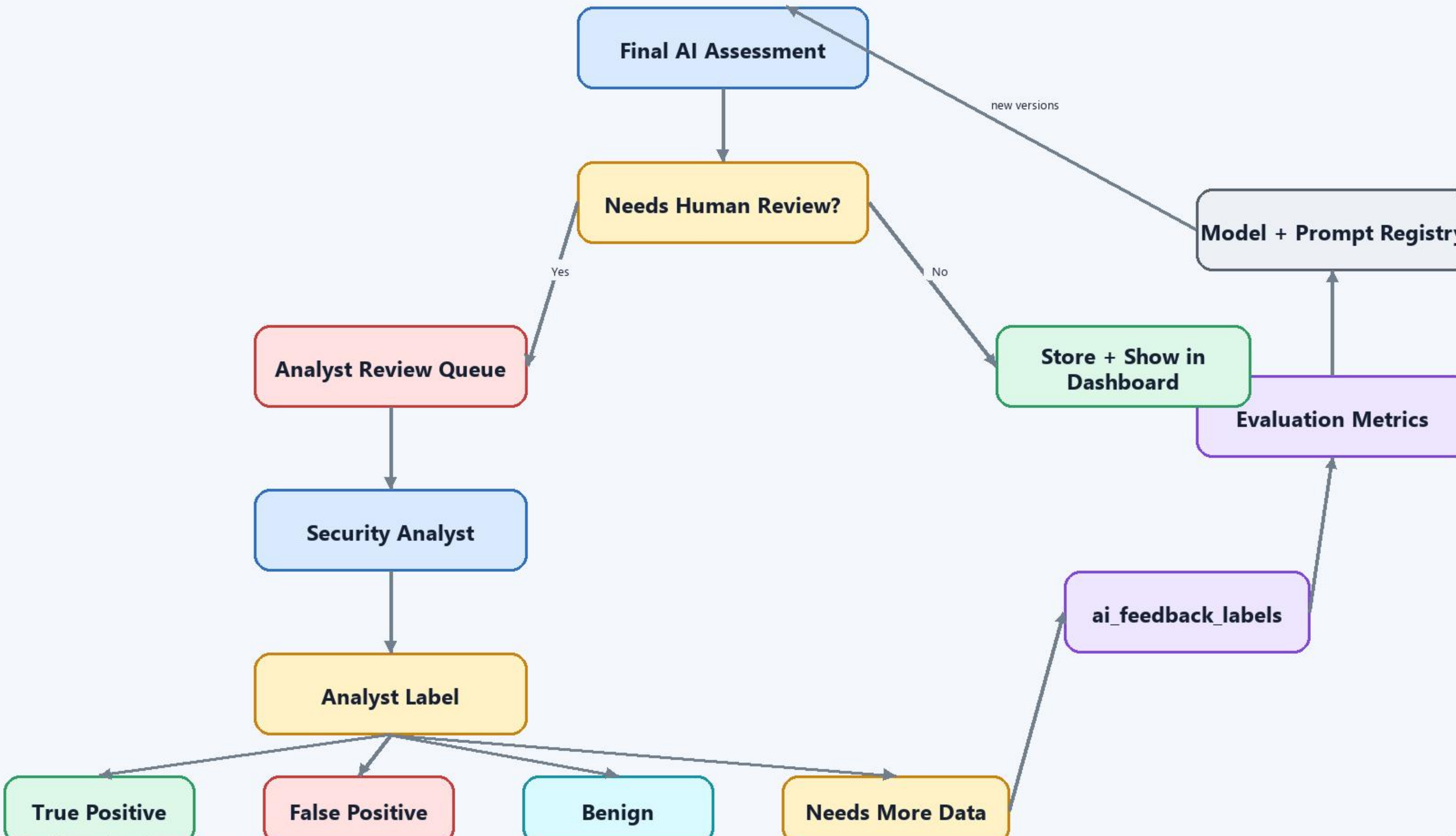


Why this matters

- The UI can show which exact evidence refs affected the final score.
- Every model run stores role, provider, model, prompt version, latency, input hash, output JSON, and status.
- Feedback can later measure model quality against analyst labels.

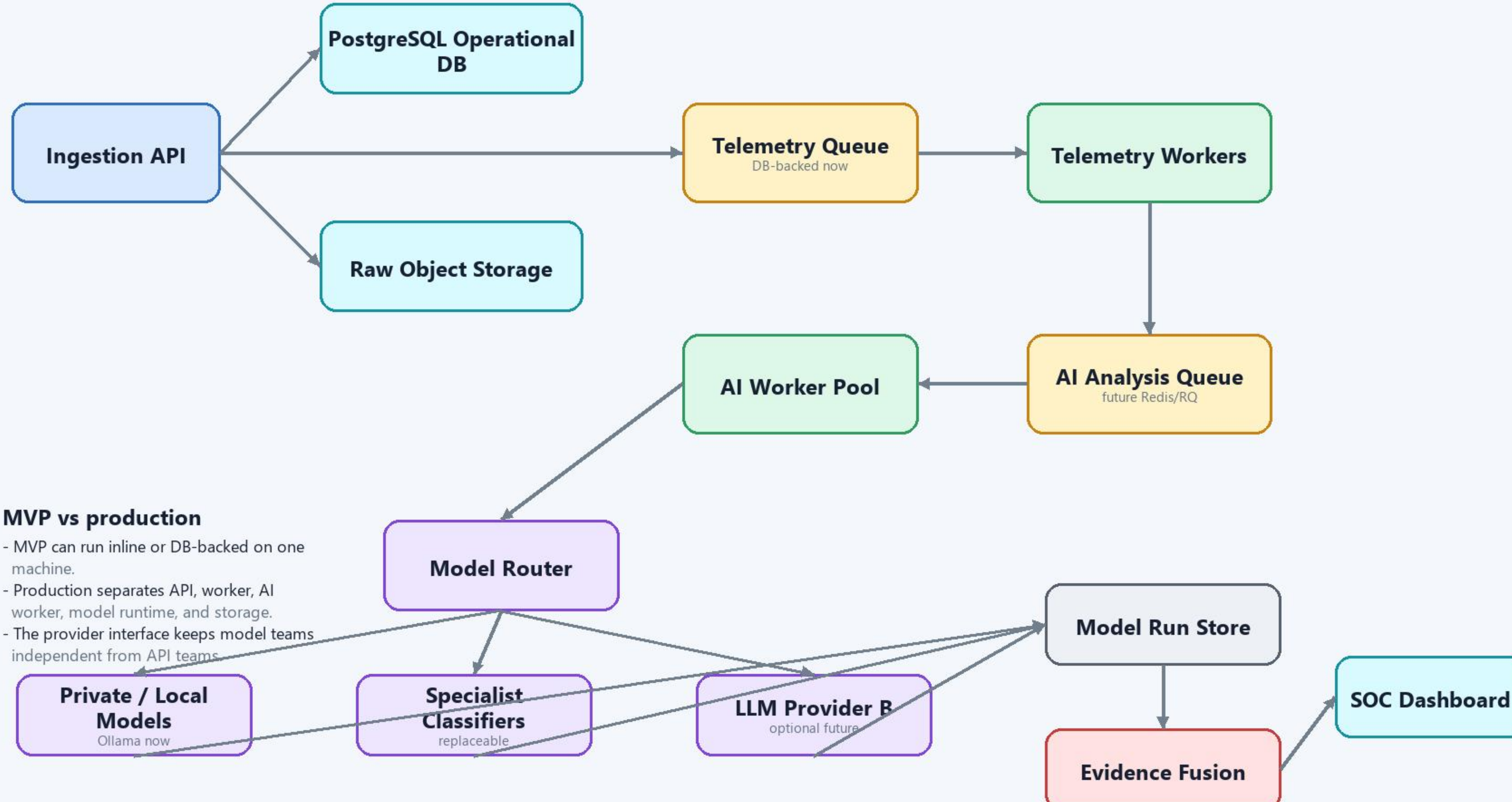
Human Review And Learning Loop

Analyst labels improve rules, prompts, models, and routing decisions



AI Deployment Topology

Keep ingestion, telemetry processing, and model analysis independently scalable



MVP vs production

- MVP can run inline or DB-backed on one machine.
- Production separates API, worker, AI worker, model runtime, and storage.
- The provider interface keeps model teams independent from API teams.

Project Plan And Phases

A practical path from local MVP to reliable security product

1. **Local Reliability** — One command starts API, React UI, Ollama. Add reset and emulator smoke scripts.
2. **Data Correctness** — Replay raw payloads, harden app snapshot semantics, add schema version and agent version.
3. **AI Quality** — Build labeled benchmark payloads, measure false positives, invalid JSON, latency, score drift.
4. **Analyst Workflow** — Improve review queue, feedback labels, finding detail, and evidence drill-down in React.
5. **Production Readiness** — Postgres Alembic migrations, external worker, retention, metrics, logs, backup policy.
6. **Security Hardening** — Per-device credentials, Play Integrity backend decode, HTTPS, RBAC, audit exports.

AI engineer work packages

- Provider engineer: improve Ollama/runtime adapter and model budgets.
- Evaluation engineer: create labeled payload/log benchmark sets.
- Prompt engineer: refine role prompts and JSON schemas.
- Data engineer: maintain lineage and feedback analytics.